

RTLinux



This article deals with installing RTLinux, and creating a sample real-time module that can be loaded in it.

An operating system is responsible for managing, accessing, controlling and interacting with the computer hardware for general and special purposes. Most common PC operating systems (Windows, Macintosh, Linux and UNIX) are general-purpose OSs, but some tasks require special-purpose operating systems that provide specific services that are not available in general-purpose OSs (GPOSSs). One such task is to obtain outputs from process execution in deterministic time frames. Such execution within a specific time bracket is known as Real-Time Processing, for which you need a Real-Time Operating System (RTOS).

In an RTOS, the basic idea is to assign the complete hardware resources to the application, and successfully execute it within a deterministic time. In GPOSS, a process might be starved of CPU resources if a higher-priority process is scheduled. For example, your music file, playing in a media player, might be kicked off from the processor if the OS needs to service an interrupt. In GPOSS, it's hard to avoid random elements; execution doesn't actually work on real-time deadlines. RTOSs take into consideration various mathematical formulas in order to execute the task; thus random elements in service availability, initialisation time and input parameters must be avoided, since they can seriously affect the real-time deadline of applications.

Many companies have worked on the development of different RTOSs, including VxWorks, Windows CE, RTLinux and QNX. RTLinux (Real-Time Linux) is what I will be discussing in the course of this article, because it incorporates a beginner-friendly, fast and flexible architecture for providing hard real-time capabilities. RTLinux, developed by Wind River, is available in two variants:

- Open RTLinux (under the GPL)
- Wind River real-time core.

We will use Open RTLinux, with the freedom of the GPL (which is also free of cost). RTLinux has patches

for both the 2.4 and 2.6 kernel series, but I would prefer the 2.4.x version, since it's been extensively tested, and is almost free of bugs.

Let's discuss the architecture of RTLinux. Figure 1 shows the traditional architecture of the general-purpose Linux system, where the Linux kernel directly interacts with the hardware. All the interrupts are intercepted by the Linux kernel itself. Figure 2 depicts the design of the RTLinux system. Three important aspects are displayed by the figure:

- RTLinux sits between the real hardware and the Linux kernel.
- It acts as the hardware for the Linux kernel.
- It treats the Linux kernel as a single big process.

The RT scheduler schedules the Linux kernel on the processor. The priority of the Linux kernel is lower compared to real-time tasks, and it can be pre-empted. All hardware interrupts are intercepted by the RTLinux layer, and if it's not for an RT task, then the interrupt is passed to the Linux kernel as a software interrupt, when RTLinux is sitting idle. Thus, two features are provided:

- All RT tasks will get a real-time response whenever they need resources.
- All non-RT tasks can be executed by the Linux kernel when the RTLinux system is idle.

Configuring your system for RTLinux

Let's now set about getting RTLinux running on a system. To test RTLinux, I have configured my system as follows:

- Pentium 4 machine running CentOS 3 (Linux 2.4.21 as the base kernel).
- An Ext2 file-system for the root filesystem.
- 2 GB of RAM

Pre-requisites

Download the Linux kernel source, and the RTLinux patch. For my chosen kernel version, that was <ftp://ftp.kernel.org/pub/linux/kernel/v2.4/linux-2.4.21.tar.bz2> and [rtlinux-3.2-wr.tar.bz2](http://www.rtlinuxfree.com/download.html) from <http://www.rtlinuxfree.com/download.html>.